

Tutorial 4

Installing C++ compiler and makefile

Overview

Revision

- Linux
- Bash commands

C++ Compiler

- Installation
- Example use

Makefiles

- Introduction and Syntax
- Execution order
- A simple makefile

Revision: Linux installation

Linux is a free, open-source operating system (OS) and Some popular distributions include: Mint and Ubuntu

- WSL (Windows Subsystem For Linux)
- Alternatives: VirtualBox

Revision: bash commands

Bash is an enhanced version of the initial Linux shell (terminal)

- File and Directory Operations
 - Commands: **ls, cd, pwd**
- File Manipulation
 - Commands: **touch, rm, mkdir, rmdir, nano**
- Archiving and Compression
 - Commands: **tar, gzip/gunzip, zip**
 - Example Zip:
 - Installation
 - `sudo apt-get install zip`
 - Extraction
 - `unzip myarchivefiles.zip`
 - Compression
 - `zip [options] [file_name.zip] [files_names]`
 - `zip [file_name.zip] [file_name]`

C++ Compiler: Installation

- Command: **sudo apt update**
 - Updates the package list from linux repository
 - Needed for upgrading or addition of new packages.
- Command: **sudo apt install build-essential**
 - Used to install Development Tools packages
 - Including gcc, g++ and make
 - GNU Compiler Collection (GCC)
 - a collection of compilers and libraries for C, C++, Objective-C, Fortran, Ada, Go , and D programming languages
- Command: **gcc --version** or **g++ --version**
 - Verify that the GCC compiler is successfully installed
 - If successful prints gcc version installed

C++ Compiler: Installation on MAC

- Command: `/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"`
 - Installs Homebrew
 - Follow the on-screen instructions to complete the installation
- Command: **brew install gcc**
 - Once Homebrew is installed, you can install GCC by running the command in Terminal.
 - This command will download and install the latest version of GCC.
- Command: **gcc --version** or **g++ --version**
 - Verify that the GCC compiler is successfully installed
 - If successful prints gcc version installed

C++ Compiler: Example use

Code:

```
#include <iostream>
using namespace std;
int main(){
    Cout << "Hello World!" << endl;
}
```

Copy code to main.cpp file

Commands:

- Compile
 - `g++ -c -std=c++98 main.cpp`
 - `g++ -std=c++98 main.o -o main`
- Execute
 - `./main`
- Clean
 - `rm *.o main`

Makefile: Introduction

Used to build an executable that might take many commands to create

- Previous example:
 - i. Compile command
 - ii. Execute command
 - iii. Clean command

How?

Save multiple commands in one file

- Set rules for executing the commands

Makefile: syntax

```
targets: prerequisites  
    command  
    command  
    command
```

- Targets
 - file names
 - when a target is run, the commands will create a file with the same name as the target
- Prerequisites
 - also file names, separated by spaces
 - files need to exist before the commands for the target are run
 - also called dependencies
- Command
 - a series of steps typically used to make the target
 - need to start with a tab character, not spaces

Makefile: example

```
main: main.o
    g++ -std=c++98 main.o -o main

main.o: main.cpp
    g++ -c -std=c++98 main.cpp

run: main
    ./main

clean: main.o main
    rm *.o main
```

Make commands:

- Compile
 - make
- Execute
 - make run
- Clean
 - make clean

End